

Algoritmi e Strutture Dati – 14/6/2024

Esercizio A1 – Complessità – Punti ≥ 8

L'equazione di ricorrenza è la seguente:

$$T(n) = \begin{cases} 1 & n < 4 \\ T(\lfloor n/2 \rfloor) + T(\lfloor n/4 \rfloor) + \lfloor n/5 \rfloor & n \geq 4 \end{cases}$$

Proviamo a dimostrare che $T(n) = \Theta(n)$. È facile vedere che $T(n) = \Omega(n)$, in quanto:

$$T(n) = T(\lfloor n/2 \rfloor) + T(\lfloor n/4 \rfloor) + \lfloor n/5 \rfloor \geq \lfloor n/5 \rfloor \geq cn$$

L'ultima disequazione è valida per $c \leq 1/5$.

Per quanto riguarda il limite superiore, proviamo con $T(n) = O(n)$. Dobbiamo quindi dimostrare che $\exists c > 0, \exists m \geq 0 : T(n) \leq cn, \forall n \geq m$. Proviamolo con il metodo di sostituzione.

Caso base

$$\begin{aligned} T(1) &= 1 \leq c \cdot 1 \Rightarrow c \geq 1 \\ T(2) &= 1 \leq c \cdot 2 \Rightarrow c \geq 1/2 \\ T(3) &= 1 \leq c \cdot 3 \Rightarrow c \geq 1/3 \\ T(4) &= T(2) + T(1) + O(1) \end{aligned}$$

Da $n = 4$ in poi non c'è bisogno di andare avanti, perché vengono generati casi già provati.

Ipotesi induttiva $\forall n' < n : T(n') \leq cn'$

Passo induttivo

$$\begin{aligned} T(n) &= T(\lfloor n/2 \rfloor) + T(\lfloor n/4 \rfloor) + \lfloor n/5 \rfloor \\ &\leq T(n/2) + T(n/4) + n/5 \\ &\leq cn/2 + cn/4 + n/5 \\ &= 3/4 \cdot cn + n/5 \\ &= n(3/4 \cdot c + 1/5) \\ &\leq cn \end{aligned}$$

L'ultima disequazione è vera per $c \geq 4/5$.

Conclusioni Mettendo insieme i limiti del caso passo e del passo induttivo, abbiamo dimostrato che $T(n) = O(n)$ per $m = 1$ e $c \geq 1$. Considerato che $T(n) = \Omega(n)$, abbiamo che $T(n) = \Theta(n)$

Esercizio A2 – Big Fire – Punti ≥ 10

Per risolvere il problema, utilizziamo una visita in profondità; avremmo potuto utilizzare una visita in ampiezza, con il vantaggio di non sovraccaricare lo stack. La procedura `bigfire()` inizializza una matrice di visite `visited` e poi chiama la funzione ricorsiva `bfRec()` per calcolare il numero di celle che prendono fuoco a partire da una determinata cella iniziale nella matrice `A`.

La funzione `bfRec()` ritorna 0 se la cella (x, y) è già stata visitata (non deve essere contata più di una volta). Altrimenti, imposta `visited[x][y]` a `true` per marcare la cella come visitata e inizializza un contatore `count` a 1 per rappresentare l'accensione della cella corrente. I due vettori `dx` e `dy` rappresentano le quattro direzioni possibili di movimento (destra, giù, sinistra, su); per ogni direzione, si calcolano le nuove coordinate (nx, ny) . Se le nuove coordinate sono all'interno dei limiti della matrice e la cella non è stata visitata; se la cella corrente è in fiamme ($A[x][y] == 1$) o è una cella vuota ($A[x][y] == 0$) adiacente a una cella in fiamme ($A[nx][ny] == 1$), la funzione ricorsivamente esplora la nuova cella (nx, ny) aggiungendo il risultato al contatore `count`. Alla fine, viene ritornato il valore del contatore `count`, che rappresenta il numero totale di celle che prendono fuoco a partire dalla cella (x, y) .

```
int bigfire(int[][] A, int n, int i, int j)
{
    boolean[][] visited = new boolean[1...n][1...n] = {false}
    return bfRec(A, n, i, j, visited)
}
```

```

int bfRec(int[][] A, int n, int i, int j, int[][] visited)
    if visited[x][y] then
        return 0
    else
        visited[x][y] = true
        int count = 1
        int[] dx = {0, 1, 0, -1}
        int[] dy = {1, 0, -1, 0}
        for i = 1 to 4 do
            int nx = x + dx[i]
            int ny = y + dy[i]
            if 1 ≤ nx ≤ n and 1 ≤ ny ≤ n and not visited[nx][ny] then
                if A[x][y] == 1 or (A[x][y] == 0 and A[nx][ny] == 1) then
                    count = count + bfRec(nx, ny)
        return count

```

Esercizio A3 – Smaller Count – Punti ≥ 12

Soluzione $O(n^2)$ Ovviamente, è banale risolvere il problema in tempo $\Theta(n^2)$, con due semplici cicli annidati.

```

int[] smaller(int[] A, int n)
    int[] count = new int[1...n]
    for i = 1 to n do
        count[i] = 0
        for j = i + 1 to n do
            if A[j] < A[i] then
                count[i] += 1
    return count

```

Soluzione $O(n \log n)$, **basata su MergeSort** Una soluzione $O(n \log n)$ è basata sul MergeSort. L'idea è questa: creiamo un dizionario *smaller* basato su tabelle hash, che associa ad ogni valore inserito k il numero di valori che sono più piccoli di k . È possibile utilizzare il valore come chiave perché abbiamo assunto che sono tutti distinti; altrimenti, si dovrebbe utilizzare la posizione del valore nel vettore e questo complicherebbe il codice. Modifichiamo la funzione `merge()` nel modo seguente: tutte le volte che selezionando il minimo tra sottovettore di sinistra e di destra si privilegia il valore del sottovettore di destra, questo valore è più piccolo di tutti i valori che si trovano a sinistra e non sono stati ancora piazzati nel vettore di appoggio. Per questo motivo, si incrementa un contatore *count* di 1. Quando si privilegia un valore k nel sottovettore di sinistra, bisogna incrementare *smaller*[k] di *count*, che contiene il numero di valori più piccoli di k che sono stati trovati durante la merge fino a questo punto.

L'operazione si ripete ricorsivamente - qualunque sia il livello, quando viene fatta una merge i valori a sinistra si trovavano prima dei valori di destra e possono essere contati correttamente da questo algoritmo.

Nel codice seguente, le modifiche rispetto all'algoritmo di MergeSort originale sono evidenziate in rosso.

```

int [] smaller(item A[], int n)
    HASH smaller = Hash()
    MergeSort(A, smaller, 1, n)
    int[] count = new int[1...n]
    for i = 1 to n do
        count[i] = smaller.lookup(A[i])
    return count

```

```
MergeSort(item A[ ], HASH smaller, int start , int end )
```

```
if start < end then
    int mid = (start + end)/2
    MergeSort(A, smaller, start, mid)
    MergeSort(A, smaller, mid + 1, end)
    Merge(A, smaller, start, end, mid)
```

```
Merge(item A[], HASH smaller, int start , int end , int mid )
```

```
int i, j, k, h
int count = 0
i = start
j = mid + 1
k = start
while i ≤ mid and j ≤ end do
    if A[i] ≤ A[j] then
        B[k] = A[i]
        int old = iff(smaller.lookup(A[i]) == nil, 0, smaller.lookup(A[i]))
        smaller.insert(A[i], old + count)
        i = i + 1
    else
        count = count + 1
        B[k] = A[j]
        j = j + 1
    k = k + 1
j = end
for h = mid downto i do
    A[j] = A[h]
    int old = iff(smaller.lookup(A[h]) == nil, 0, smaller.lookup(A[h]))
    smaller.insert(A[h], old + 1)
    j = j - 1
for j = start to k - 1 do
    A[j] = B[j]
```

Soluzione $O(n \log n)$, basata su alberi red-black Menziono questa soluzione perché interessante, ma non fornisco codice perché richiede parecchie modifiche agli alberi redblack. La soluzione consiste nel mantenere in ogni nodo x dell'albero il numero di nodi dell'albero contenuti nel sottoalbero radicato in x . In un albero binario è facilmente implementabile inserendo un contatore in ogni nodo e aumentandolo di uno tutte le volte che si attraversa un nodo durante le operazioni di inserimento. Sfortunatamente, in un albero redblack bisogna anche modificare tutte le operazioni di bilanciamento e questo è troppo complesso.

Assumendo di avere questi valori, quando si inserisce un valore il numero di valori che sono più piccoli di quello è pari a zero se viene inserito come figlio sinistro del parent, è pari al numero di nodi nel sottoalbero sinistro del parent più uno (il parent stesso). La procedura di inserimento potrebbe restituire questo valore oltre all'albero. A questo punto, si inseriscono i valori a partire dall'ultimo: quando viene inserito il valore, il numero di nodi minori è pari al valore restituito dall'operazione insert.

Esercizio B1 – Corsa a ostacoli – Punti ≥ 8

Questo problema può essere risolto tramite programmazione dinamica. Scriviamo un'equazione di ricorrenza che restituisca il numero di percorsi per andare dalla cella (i, j) alla cella $(1, 1)$.

$$DP[i][j] = \begin{cases} 0 & A[i][j] = \text{false} \vee i == 0 \vee j = 0 \\ 1 & i = 1 \wedge j = 1 \wedge A[i][j] = \text{true} \\ DP[i-1][j] + DP[i][j-1] & \text{altrimenti} \end{cases}$$

In altre parole, il numero di percorsi che passano per una cella con valore **false** è 0; il numero di percorsi che "sfiorano" in alto e a sinistra è 0; il numero di percorsi a partire dalla cella $(1, 1)$ è pari a 1; il numero di percorsi a partire da qualunque altra cella è pari al numero di percorsi andando a destra e in basso.